

UNIVERSIDAD DE LAS FUERZAS ARMADAS – ESPE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

CARRERA DE INGENIERÍA EN SOFTWARE

ASIGNATURA: DESARROLLO DE VIDEOJUEGOS

UNIDAD 1

Tres en Raya con Inteligencia Artificial

Proyecto 3: Tic-Tac-Toe (UnbeatableAI + MediumAI) – Go (Web) + Angular

---

DOCENTE: ING. CÉSAR VILLACÍS SILVA, MSC.

ESTUDIANTES:

Diana Guerra
Paul Moreno
Germán Cáceres

GRUPO N.º 3

FECHA: NOVIEMBRE 2026

Índice

1. Proyectos Propuestos del Capítulo 1 **2**

2. Especificación de Requerimientos (IEEE 830) **2**
2.1. Introducción . . . 2
2.1.1. Propósito . . . 2
2.1.2. Alcance . . . 2
2.1.3. Definiciones, acrónimos y abreviaturas . . . 3
2.1.4. Referencias . . . 3
2.2. Descripción general . . . 3
2.2.1. Perspectiva del producto . . . 3
2.2.2. Funciones del producto . . . 3
2.2.3. Características de los usuarios . . . 3
2.2.4. Restricciones . . . 3
2.2.5. Suposiciones y dependencias . . . 4
2.3. Requerimientos específicos . . . 4
2.3.1. Requerimientos funcionales . . . 4
2.3.2. Requerimientos no funcionales . . . 4

3. Modelo de Inteligencia Artificial **5**
3.1. Variante MediumAI . . . 6

4. Diagramas **7**
4.1. Diagrama de casos de uso . . . 7
4.2. Diagrama de secuencia: marcar casilla . . . 8
4.3. Diagrama de clases (backend Go) . . . 9

5. Metodología SCRUM + XP **9**
5.1. Roles . . . 9
5.2. Product Backlog . . . 10
5.3. Sprints . . . 10
5.4. Prácticas XP aplicadas . . . 10
5.5. Diseño . . . 11
5.6. Desarrollo . . . 11
5.7. Pruebas . . . 12

6. Análisis de Resultados: Métricas de Software **12**
6.1. Líneas de código . . . 12
6.2. Otras métricas . . . 13
6.3. Hallazgos sobre la lógica del docente . . . 13

7. Conclusiones y Recomendaciones **13**
7.1. Conclusiones . . . 13
7.2. Recomendaciones . . . 14

1. Proyectos Propuestos del Capítulo 1

El docente propuso implementar el juego del Tres en Raya con niveles de dificultad en distintas tecnologías. Al Grupo 3 le corresponden los proyectos web en Go y JavaScript:

Tabla 1: Proyecto 3 – Grupo 3

N.º	Variante de IA	Tecnología	Estado
1	UnbeatableAI	Go (Web) + Angular	**Implementado**
2	MediumAI	Go (Web) + Angular	**Implementado**
3	UnbeatableAI	JavaScript (Web)	–
4	MediumAI	JavaScript (Web)	–
5	UnbeatableAI → MinMax	Go (Web) + Angular	Apartado preparado
6	UnbeatableAI → MinMax	JavaScript (Web)	–

El equipo eligió **Go** para el backend y **Angular** para el frontend. Por especificación, la lógica del juego es *exactamente la misma* que la de la versión oficial del docente (`Tic-Tac-Toe-Game-Windows-Professor`); solo cambia la tecnología. Esa versión trae una aplicación de Windows Forms por variante:

Tabla 2: Versiones de referencia del docente

Variante	Carpeta	Clase
UnbeatableAI	`Tic-Tac-Toe-Versión-IA-Fuerte`	`UnbeatableAIController`
MediumAI	`Tic-Tac-Toe-Versión-IA-Debil`	`MediumAIController`
MinMax	`Tic-Tac-Toe-Version-MiniMax`	`UnbeatableAIController`*

*Misma clase, pero implementada con el algoritmo Minimax en lugar del método numérico.

2. Especificación de Requerimientos (IEEE 830)

2.1. Introducción

2.1.1. Propósito

Este documento especifica los requerimientos del videojuego educativo *Tres en Raya con IA* en su versión web. Va dirigido al docente de la asignatura y a los integrantes del grupo.

2.1.2. Alcance

El sistema permite que un usuario juegue al Tres en Raya contra un *Jugador Controlado por la Computadora* (JCC) desde un navegador. La JCC decide sus jugadas con el método numérico heurístico de la referencia (pesos usuario = 1, JCC = 3, vacío = 0). Los niveles implementados son **UnbeatableAI** y **MediumAI**; **UnbeatableAI → MinMax** permanece preparado.

2.1.3. Definiciones, acrónimos y abreviaturas

JCC Jugador Controlado por la Computadora (marca "O").

Usuario Jugador humano (marca "X"); siempre empieza.

mArrayMoves
Vector V_k de 9 posiciones con los pesos 0/1/3.

API REST Interfaz HTTP/JSON entre el frontend y el backend.

SPA *Single Page Application*, aplicación web de una sola página.

2.1.4. Referencias

- IEEE Std 830-1998, *Recommended Practice for Software Requirements Specifications.*
- Villacís, C. et al., *Optimización del Juego del Tres en Raya con Niveles de Dificultad, Utilizando Técnicas Heurísticas de IA*, ESPE.
- Material de clase: *Caso 1*, *Caso 2* y *Caso 3 TTT.*
- Código del docente: `Tic-Tac-Toe-Game-Windows-Professor` (IA fuerte, IA débil y MiniMax).

2.2. Descripción general

2.2.1. Perspectiva del producto

Es una aplicación cliente–servidor. El **backend en Go** contiene *toda* la lógica del juego (port 1:1 de las clases C#) y la expone como API REST. El **frontend en Angular** solo dibuja el tablero, envía los clics y anima los eventos que devuelve el servidor.

2.2.2. Funciones del producto

Seleccionar modo de juego, iniciar una partida nueva, marcar una casilla, recibir la jugada de la JCC, conocer el resultado ("¡Ganó X!", "¡Ganó O!" o "¡Empate!"), ver el marcador y ver cómo razona la IA (pesos y sumas por línea).

2.2.3. Características de los usuarios

Niños de 7 a 11 años (público objetivo del juego original) y estudiantes. No se requiere conocimiento técnico: basta con hacer clic o usar las teclas 1–9.

2.2.4. Restricciones

- La lógica del juego no puede modificarse respecto al original.
- Backend en Go 1.23 o superior, solo con la librería estándar.
- Frontend en Angular 22 con navegadores modernos (Chrome, Edge, Firefox).

2.2.5. Suposiciones y dependencias

El servidor Go está disponible en el puerto 8080; en desarrollo, Angular redirige `/api` mediante `proxy.conf.json`.

2.3. Requerimientos específicos

2.3.1. Requerimientos funcionales

Tabla 3: Requerimientos funcionales

ID	Descripción	Prioridad
RF01	El sistema mostrará un tablero de 3×3 casillas numeradas 0–8 (TabIndex).	Alta
RF02	El usuario podrá seleccionar el modo de juego; los modos no implementados se mostrarán como "en construcción".	Alta
RF03	El usuario siempre inicia con la marca "X".	Alta
RF04	Al marcar una casilla libre, el sistema registrará peso 1 en `mArrayMoves`; una casilla ocupada se ignora.	Alta
RF05	En su primera jugada la JCC tomará el centro (4) si está libre; si no, una esquina aleatoria (0, 2, 6, 8). En las jugadas siguientes, si el centro está libre, lo toma.	Alta
RF06	La JCC aplicará el método numérico: suma 6 ⇒ gana; suma 2 ⇒ bloquea; suma 5 en diagonal ⇒ caso trivial; dos lados adyacentes del usuario ⇒ esquina intermedia (orden 1+3→0, 1+5→2, 3+7→6, 5+7→8); en otro caso, casilla aleatoria.	Alta
RF07	Si la JCC está en el centro y la regla de suma 5 aplica, jugará un lado aleatorio entre 1, 5 y 7 (comportamiento del código del docente, ver sección 6.3).	Alta
RF08	El sistema verificará ganador recorriendo las 8 líneas (filas, columnas, diagonales); con 9 jugadas sin ganador declara empate. Mensajes: "¡Ganó X!", "¡Ganó O!", "¡Empate!".	Alta
RF09	Al terminar la partida el tablero se deshabilitará (equivalente a `grbTicTacToe.Enabled = false`).	Alta
RF10	El usuario podrá iniciar un juego nuevo en cualquier momento.	Media
RF11	El sistema mostrará un marcador de partidas ganadas, empatadas y perdidas por modo.	Media
RF12	El sistema mostrará los pesos de `mArrayMoves` y la suma de cada línea (inspector de IA).	Baja

2.3.2. Requerimientos no funcionales

Tabla 4: Requerimientos no funcionales

ID	Tipo	Descripción
RNF01	Fidelidad	La lógica de la IA será idéntica a `UnbeatableAIController.cs` (versión IA Fuerte del docente).
RNF02	Rendimiento	El servidor responderá a cada jugada en menos de 50 ms.
RNF03	Usabilidad	Interfaz animada, con sonido opcional y soporte de teclado (1–9 y teclado numérico).
RNF04	Accesibilidad	Respeta `prefers-reduced-motion` y usa etiquetas ARIA en el tablero.
RNF05	Portabilidad	Funciona en navegadores modernos y es adaptable a celular, tablet y escritorio.
RNF06	Mantenibilidad	Cada modo vive en su propio archivo y cumple una interfaz común (`Controller`).
RNF07	Fiabilidad	Ninguna petición puede bloquear el servidor (protección ante bucles infinitos del original).
RNF08	Concurrencia	Varias partidas simultáneas, aisladas por sesión, con exclusión mutua.
RNF09	Calidad	Pruebas automatizadas con `go test` para los Casos 1, 2 y 3.

3. Modelo de Inteligencia Artificial

El tablero se representa como el vector V_k, con $k = 0, \ldots, 8$, en el arreglo `mArrayMoves`. Cada casilla vale 0 (vacía), 1 (usuario) o 3 (JCC). Para cada una de las 8 líneas se calcula la suma de sus tres casillas:

Tabla 5: Interpretación de la suma por línea

Suma	Composición	Acción de la JCC
6	$3 + 3 + 0$	Completar la línea y **ganar** (Caso 2)
2	$1 + 1 + 0$	**Bloquear** al usuario (Caso 1)
5 (diagonal)	$1 + 3 + 1$	Caso trivial: esquina libre o lado aleatorio (Caso 3)

La regla de bloqueo en Go conserva el mismo orden de evaluación que el C#:

```go
// pick devuelve el primer casillero vacío de la línea (a, b, d).
func (c *UnbeatableAIController) pick(a, b, d int) int {
       index := 0
       if c.ArrayMoves[a] == 0 { index = a
       } else if c.ArrayMoves[b] == 0 { index = b
       } else if c.ArrayMoves[d] == 0 { index = d }
       c.heuristicSum = c.ArrayMoves[a] + c.ArrayMoves[b] + c.ArrayMoves[d]
       return index
}
```

3.1. Variante MediumAI

`MediumAIController` porta la versión de IA débil del docente y conserva sus pesos: 0 para una casilla vacía, 1 para X y 3 para O. El turno humano siempre inicia con X. Después de cada jugada válida, el controlador comprueba las ocho líneas ganadoras y el empate; al terminar, registra el mensaje del resultado y deshabilita el tablero.

La JCC (O) aplica las reglas en este orden:

1. Si el centro (`CenterIndex = 4`) está vacío, lo elige de inmediato.
2. Si el centro está ocupado, recorre filas, columnas y diagonales en el orden de `WinningLines`. Primero busca una suma de 6 $(3 + 3 + 0)$ para completar una línea y ganar.
3. Si no hay una suma de 6, busca una suma de 2 $(1 + 1 + 0)$ para bloquear la primera amenaza de X, eligiendo la primera casilla vacía de esa línea.
4. Si ninguna regla aplica, elige al azar una casilla vacía.

La prioridad global de suma 6 antes de suma 2 permite que O gane cuando tiene una jugada ganadora disponible, aunque exista una amenaza de X. Esta variante no incluye las reglas de suma 5 ni la estrategia de esquinas y lados de `UnbeatableAI`. El backend expone los pesos por medio de `Weights()` y comunica jugadas y resultados como eventos del tablero; la interfaz Angular los presenta sin decidir movimientos.

4. Diagramas

4.1. Diagrama de casos de uso

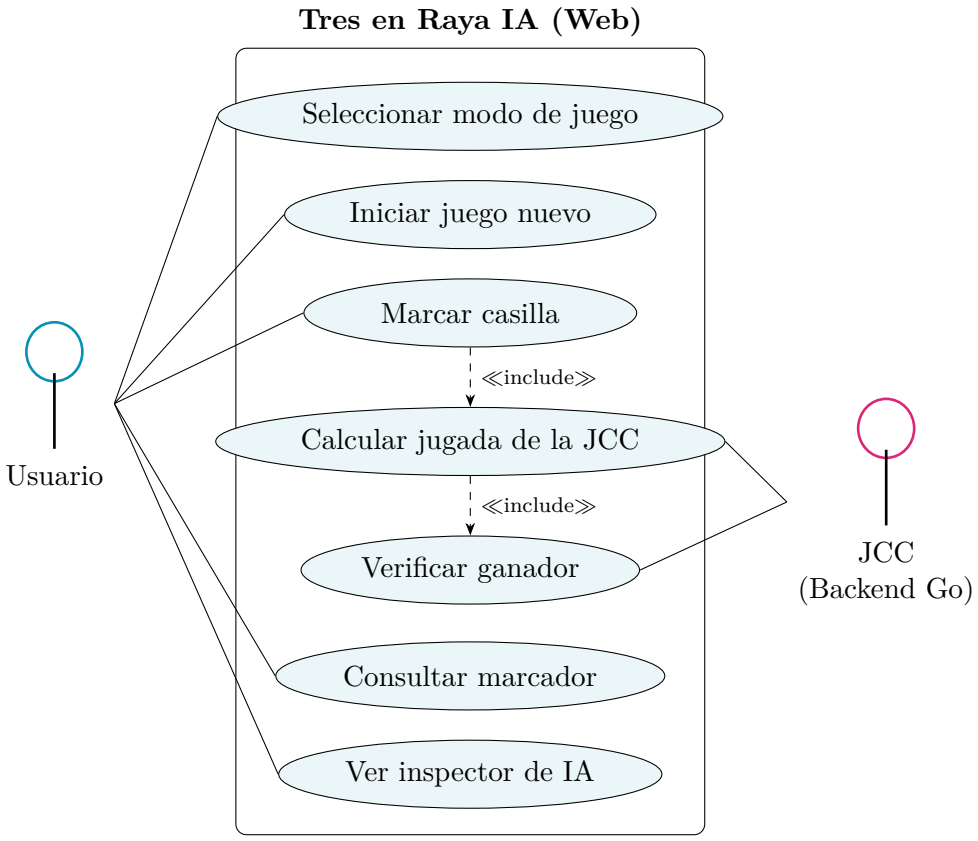


Figura 1: Diagrama de casos de uso

4.2. Diagrama de secuencia: marcar casilla

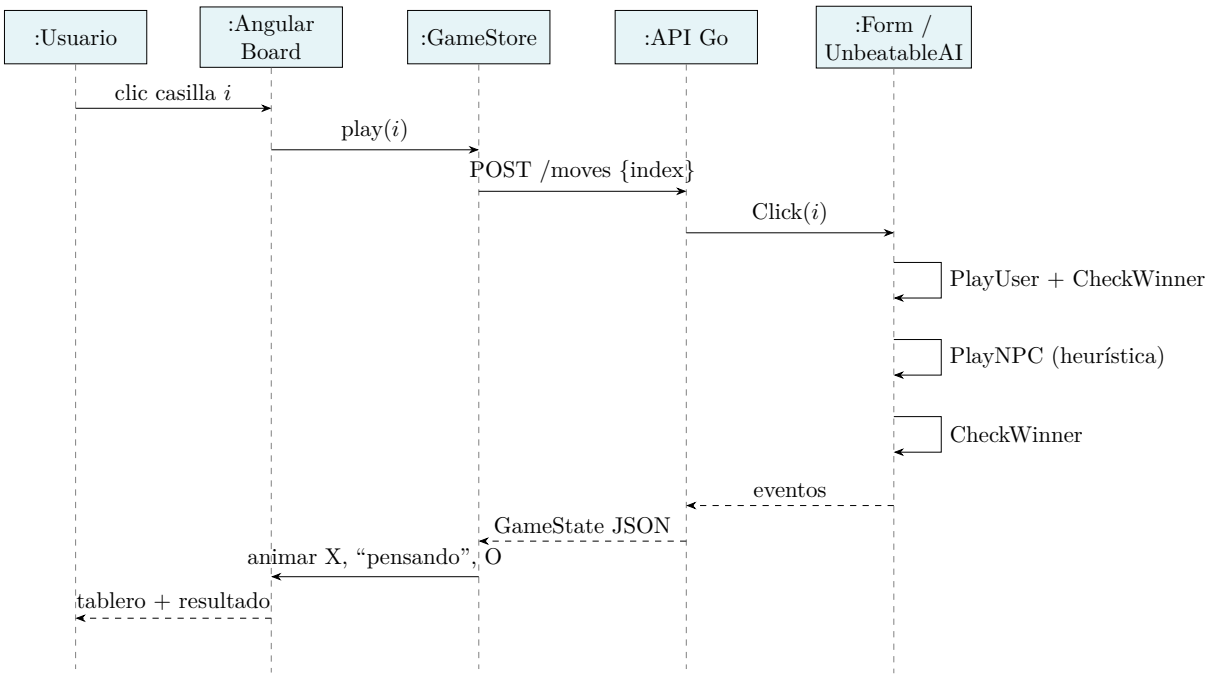


Figura 2: Diagrama de secuencia del caso de uso "Marcar casilla". Si hay ganador o empate, `Board.Enabled` pasa a `false` y el frontend muestra el resultado con animación.

4.3. Diagrama de clases (backend Go)

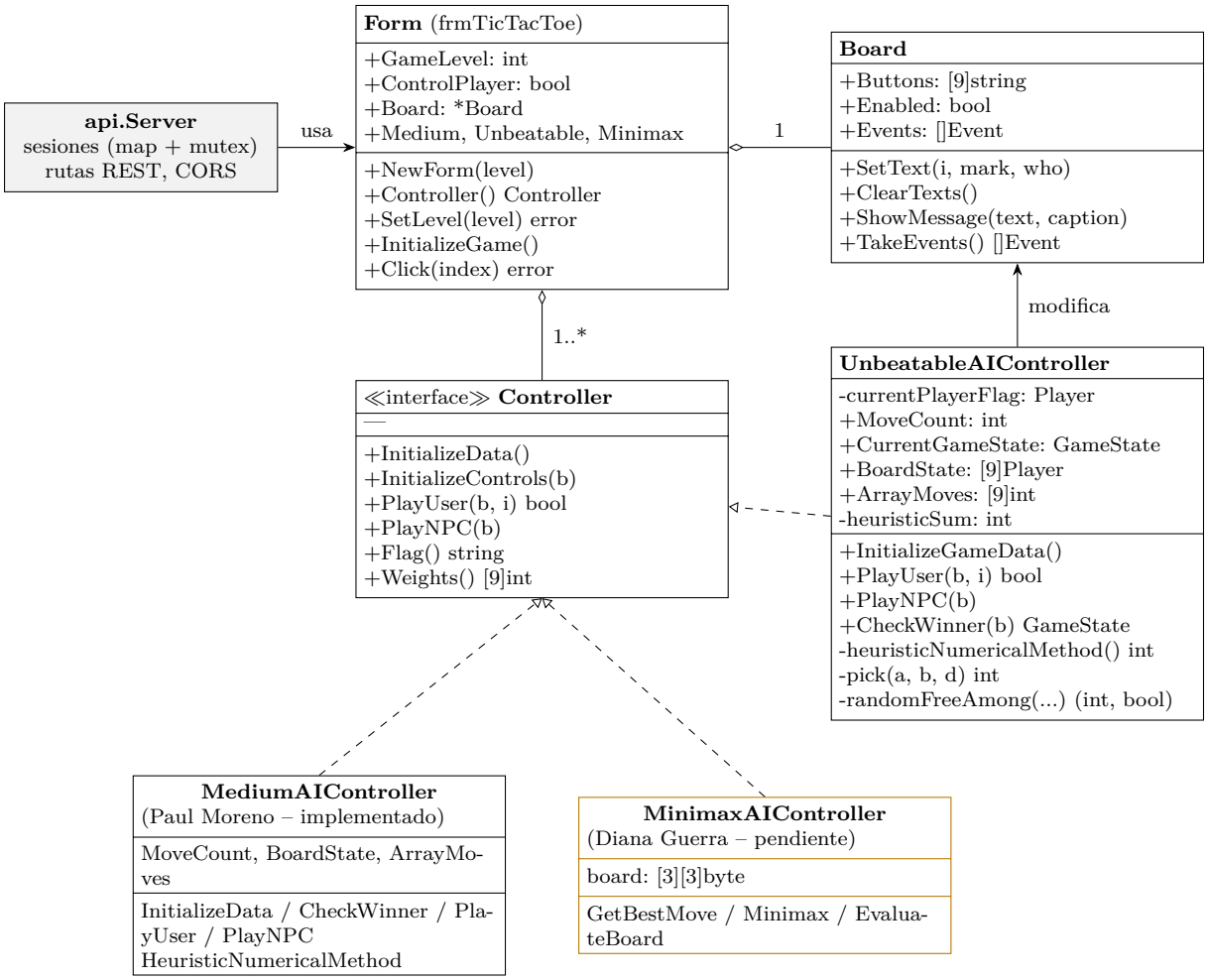


Figura 3: Diagrama de clases del backend

5. Metodología SCRUM + XP

Se combinó **SCRUM** para organizar el trabajo en *sprints* y **XP** para las prácticas de ingeniería.

5.1. Roles

Tabla 6: Roles del equipo

Rol	Responsable
Product Owner	Ing. César Villacís Silva, MSc.
Scrum Master	Diana Guerra
Desarrollo – UnbeatableAI (Go + Angular)	Germán Cáceres
Desarrollo – MediumAI	Paul Moreno
Desarrollo – UnbeatableAI → MinMax	Diana Guerra

5.2. Product Backlog

Tabla 7: Historias de usuario

HU	Historia	Pts	Sprint
HU01	Como usuario quiero ver un tablero 3×3 para poder jugar.	3	1
HU02	Como usuario quiero marcar una casilla y que la JCC responda.	5	1
HU03	Como usuario quiero enfrentar a una IA que gane y bloquee (UnbeatableAI).	8	1
HU04	Como usuario quiero saber quién ganó o si hubo empate.	3	1
HU05	Como usuario quiero elegir el modo de juego.	3	2
HU06	Como usuario quiero animaciones y sonidos para que el juego sea atractivo.	5	2
HU07	Como estudiante quiero ver los pesos y sumas que usa la IA.	3	2
HU08	Como usuario quiero un marcador de partidas.	2	2
HU09	Como usuario quiero jugar contra MediumAI.	5	3
HU10	Como usuario quiero jugar contra UnbeatableAI con MinMax.	8	3

5.3. Sprints

Sprint 1 – Núcleo del juego. Port del método numérico a Go (`unbeatable_ai.go`), tablero (`board.go`), despacho de clics (`form.go`), API REST y pruebas de los Casos 1–3.

Sprint 2 – Experiencia web. Frontend Angular 22: tablero 3D con trazos animados, fondo de partículas, sonidos sintetizados, inspector de IA, marcador y pantalla de resultado con confeti.

Sprint 3 – MediumAI. Port del controlador de IA débil e integración del modo en la interfaz `Controller`.

Pendiente: Completar y documentar el modo MinMax.

5.4. Prácticas XP aplicadas

- **Diseño simple:** un archivo por modo y una interfaz común.
- **Pruebas primero:** los ejemplos de los PDF se escribieron como pruebas (`unbeatable_ai_test.go`) antes de conectar la interfaz.
- **Estándar de código:** `gofmt`/`go vet` en Go y Prettier en Angular.
- **Integración continua y entregas pequeñas:** cada incremento se sube al repositorio `github.com/Gpcaceres/videojuegos-grupo3`.
- **Propiedad colectiva:** el `README` explica cómo completar cada modo.

5.5. Diseño

La arquitectura separa responsabilidades en dos capas:

- **Backend (Go):** `internal/game` contiene la lógica (sin dependencias web) e `internal/api` expone las rutas. `Board` reemplaza a los controles de Windows Forms (`ListButtons`, `GroupBox` y `MessageBox`), de modo que el código de los controladores queda casi línea por línea igual al original.
- **Frontend (Angular):** componentes *standalone* con *signals*, sin `zone.js`. `GameStore` reproduce los eventos del backend con pausas para animarlos y no decide ninguna jugada.

Tabla 8: API REST

Método	**Ruta**	**Equivale a (original)**
GET	/api/modes	Elegir qué aplicación (variante) abrir
POST	/api/games	Abrir el formulario (`frmTicTacToe_Load`)
POST	/api/games/{id}/moves	`btnTicTacToeN_Click`
POST	/api/games/{id}/new	`btnNew_Click`
PUT	/api/games/{id}/level	Cambiar de variante

5.6. Desarrollo

La correspondencia entre el original y el port es directa:

Tabla 9: Correspondencia C# → Go

C# (original)	**Go**
`ListButtons[i].Text`	`Board.Buttons[i]`
`grbTicTacToe.Enabled`	`Board.Enabled`
`MessageBox.Show(...)`	`Board.ShowMessage(...)` → evento
`enum Player` / `enum GameState`	`type Player int` / `type GameState int`
`mBoardState` (`Player[]`)	`BoardState [9]Player`
`mArrayMoves` (`int[]`)	`ArrayMoves [9]int`
`CENTER_INDEX`, `CORNER_/SIDE_INDICES`	`CenterIndex`, `Corner/SideIndices`
`new Random().Next(0, 9)`	`rand.Intn(9)`
`mHeuristicSum` (miembro)	`heuristicSum` (se conserva entre turnos)
`static bool ControlPlayer`	`Form.ControlPlayer` (por sesión)

La única diferencia deliberada es la protección de los `do/while` aleatorios: si no queda ninguna casilla del grupo pedido, el original entra en un bucle infinito. En el servidor esto bloquearía la petición, así que se continúa con la jugada heurística (RNF07).

5.7. Pruebas

Tabla 10: Resultados de `go test ./...`

Prueba	Verifica	Resultado
TestFirstMoveTakesCenter	La JCC toma el centro en su primera jugada	PASS
TestFirstMoveCornerWhenCenterTaken	Si el centro está ocupado, toma una esquina (50 repeticiones)	PASS
TestCase1Blocks	Caso 1: bloquea al usuario (suma 2)	PASS
TestCase2Wins	Caso 2: completa la línea y gana (suma 6)	PASS
TestTakesImmediateWin	Prioriza ganar en la diagonal	PASS
TestAdjacentSidesCornerOrder	Orden oficial de las esquinas (1+3→0 antes que 1+5→2)	PASS
TestSideRuleNeverPicksThree	Con suma 5 y JCC en el centro, elige lado 1, 5 o 7, nunca 3 (200 repeticiones)	PASS
TestOriginalLogicExhaustive	Recorre todas las partidas posibles	PASS

Además se verificó la aplicación completa en el navegador (front + back): la JCC responde en el centro, el resultado se muestra al terminar y los modos pendientes aparecen bloqueados.

Pendiente: agregar capturas de pantalla de la aplicación en `Img/` (por ejemplo `captura-juego.png`).

6. Análisis de Resultados: Métricas de Software

6.1. Líneas de código

Se contaron las líneas efectivas de código (sin comentarios ni líneas en blanco):

Tabla 11: Líneas de código por proyecto

Proyecto	Archivos	LOC	Coment.	Blanco
Referencia del docente, IA Fuerte (C#)	3	559	87	56
Backend Go (este proyecto)	9	854	156	116
Frontend Angular (TS + HTML + SCSS)	24	2 179	36	292

Tabla 12: Clase de la IA: original frente al port

Archivo	LOC	Funciones/métodos
UnbeatableAIController.cs (docente)	417	15
unbeatable_ai.go	**223**	**17**

El port en Go reduce un 47 % las líneas de la clase de IA frente al C#. Las mismas reglas se expresan con un `switch` sin repetir bloques, y una función auxiliar `pick` reemplaza

los 16 bloques `if/else if` iguales (uno por línea y por suma). Las constantes y enumeraciones compartidas (`Player`, `GameState`, `WinningLines`) quedan en `controller.go` para que MediumAI las reutilice.

6.2. Otras métricas

- **Pruebas automatizadas:** 8 de 8 aprobadas; la exploración exhaustiva cubre alrededor de 15 000 partidas en 0,02 s.
- **Tamaño del frontend:** 216 kB sin comprimir (62 kB transferidos) en el *build* de producción.
- **Dependencias del backend:** cero (solo librería estándar de Go).
- **Tiempo de respuesta:** menos de 1 ms por jugada en el servidor local.

6.3. Hallazgos sobre la lógica del docente

Al portar y probar `UnbeatableAIController.cs` se encontró lo siguiente. Como la especificación exige conservar la lógica, todo se mantiene igual y se documenta aquí.

1. **No es 100 % imbatible.** La prueba exhaustiva muestra que la JCC **pierde en cerca del 1,1 % de los caminos posibles**. Cuando ninguna regla aplica, la lógica cae en `GenerateRandomNPCMove` y a veces deja al usuario un *tenedor* (dos amenazas a la vez). Ejemplo: X en 0, O en 4, X en 7, O en 2 (al azar), X en 6. La JCC solo puede tapar una de las amenazas (0–3–6 y 6–7–8), y el usuario gana.
2. **El lado 3 nunca se elige.** En `PlayNPC`, la condición del lado aleatorio compara `SIDE_INDICES[0]` dos veces (`index == SIDE_INDICES[0] || index == SIDE_INDICES[0] || ...`), así que el lado 3 queda fuera. Se reprodujo tal cual y la prueba `TestSideRuleNeverPicksThree` lo verifica.
3. **Posible bloqueo.** Los `do/while` aleatorios no tienen salida si no queda ninguna casilla válida del grupo. En las cerca de 15 000 partidas exploradas eso **no ocurrió nunca**, pero el servidor tiene una protección (RNF07) por si acaso.
4. **Regla del centro sin efecto.** A partir de la segunda jugada, `PlayNPC` revisa si el centro está libre; nunca lo está a esa altura, porque la primera jugada de alguno de los dos ya lo ocupó.

El primer punto es justamente la motivación de la variante **UnbeatableAI → MinMax**, que explora el árbol completo de jugadas y no puede caer en un tenedor.

Pendiente: Agregar métricas de MinMax (LOC, pruebas y porcentaje de derrotas) cuando esté implementado.

7. Conclusiones y Recomendaciones

7.1. Conclusiones

- Se migró `UnbeatableAIController` (versión IA Fuerte del docente) de C# a Go sin alterar su lógica. Las pruebas reproducen los Casos 1, 2 y 3 del material de clase.

- Se integró MediumAI con prioridad al centro, reglas de suma 6 y 2, y selección aleatoria como respaldo, conservando la lógica del docente.
- Separar la lógica (backend) de la presentación (frontend) permitió sustituir Windows Forms por una interfaz web animada sin tocar el algoritmo.
- El método numérico basado en pesos 1/3/0 es simple y eficiente, pero no es estrictamente imbatible: su caída al azar deja un 1,1 % de caminos perdedores.

7.2. Recomendaciones

- Implementar la variante MinMax para garantizar una IA realmente imbatible.
- Agregar las reglas de "bloquear tenedor" si en el futuro se permite modificar la heurística.
- Desplegar el backend en un servicio en la nube (por ejemplo Cloud Run) y el frontend como sitio estático.

Referencias

[1] IEEE, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998.

[2] C. Villacís, W. Fuertes, A. Bustamante et al., *Optimización del Juego del Tres en Raya con Niveles de Dificultad, Utilizando Técnicas Heurísticas de Inteligencia Artificial*, Universidad de las Fuerzas Armadas – ESPE.

[3] The Go Authors, *The Go Programming Language Documentation*, `https://go.dev/doc/`.

[4] Google, *Angular Documentation*, `https://angular.dev`.

[5] K. Schwaber y J. Sutherland, *La Guía de Scrum*, 2020.

[6] K. Beck, *Extreme Programming Explained: Embrace Change*, Addison-Wesley, 2004.